

# timseil.dev

Wie diese Seite funktioniert —  
Konzept, Architektur, Mechanik, Betrieb

Jede Behauptung der Seite ist an einen Beleg gebunden,  
und der Beleg ist ein System, das läuft.

|         |                                              |
|---------|----------------------------------------------|
| DOMAIN  | timseil.dev                                  |
| STACK   | Next.js 16.3 LTS • Go 1.26 • PostgreSQL 18.6 |
| BETRIEB | OVH VPS • Dokploy • Traefik • GitHub Actions |
| MESSUNG | Alloy • Prometheus • Loki • Grafana          |
| STAND   | 16.08.2026                                   |

# Inhalt

## TEIL I – DAS KONZEPT

---

- 1 · Die These
  - 2 · Wer liest diese Seite, und was prüft er
  - 3 · Die neun Invarianten
  - 4 · Terminal Noir — warum diese Ästhetik
- 

## TEIL II – DIE ARCHITEKTUR

---

- 5 · Systemüberblick
  - 6 · Wer welche Daten besitzt
  - 7 · Der Messweg
  - 8 · Das externe Ausfallprotokoll
  - 9 · Was bewusst fehlt
- 

## TEIL III – DIE MECHANIK

---

- 10 · Das Datenmodell
  - 11 · Die Ableitung
  - 12 · Metriken — nullable by design
  - 13 · Das Betriebsraster
  - 14 · Der Contract
  - 15 · Der Contribution-Graph
  - 16 · Das Kontaktformular
  - 17 · Das Terminal
  - 18 · Das Error-Budget-Spiel
- 

## TEIL IV – DIE OBERFLÄCHE

---

- 19 · Tokens und das Theme-System
  - 20 · Layout und die sieben Prüfbreiten
  - 21 · Die Boot-Sequenz
-

22 · Scroll-Choreografie

---

23 · Zustandssprache

---

24 · Barrierefreiheit

---

## TEIL V – DER BETRIEB

---

25 · Container und Compose

---

26 · CI/CD

---

27 · Observability

---

28 · SLOs und Fehlerbudget

---

29 · Sicherheit

---

30 · Datenschutz

---

## TEIL VI – REFERENZ

---

A · Endpoint-Referenz · B · Zustands-Referenz

---

C · Glossar · D · Entscheidungsregister

---

Nachwort

---

# TEIL I – DAS KONZEPT

---

## 1. Die These

---

Jede Portfolioseite behauptet etwas. Die meisten behaupten es mit Screenshots, Logolisten und Prozentbalken: „React 90 %, Docker 75 %.“ Wer das liest, muss es glauben — prüfen kann er nichts. Der Balken ist eine Selbsteinschätzung, die als Messung auftritt.

Diese Seite macht das Gegenteil. Sie hat genau einen Satz als Fundament:

Jede Behauptung der Seite ist an einen Beleg gebunden, und der Beleg ist ein System, das läuft.

Daraus folgt alles Weitere. Es gibt keine Prozentbalken, weil Prozente nichts belegen. Es gibt Zustände — `CORE`, `APPLIED`, `LEARNING`, `QUEUED` — und unter jedem steht, aus welchem laufenden System er abgeleitet ist. Es gibt keine erfundenen Betriebszahlen, weil eine erfundene Zahl in dreißig Sekunden auffliegt. Wo keine Messung vorliegt, steht `– NO DATA`.

Und es gibt eine öffentliche API. Wer der Seite nicht glaubt, tippt:

```
curl https://timseil.dev/api/systems
```

und bekommt dieselben Zahlen, die auf der Seite stehen. Das ist der Prüfstein. Ohne ihn wäre die These eine weitere Behauptung; mit ihm ist sie ein Angebot.

### Warum das ein besseres Argument ist

Ein Hiring Manager, der zwanzig Portfolios am Tag sieht, hat gelernt, Selbstauskünfte zu ignorieren. Er kann sie nicht prüfen, also gewichtet er sie mit null. Was er kann, ist eine Sache anschauen, die läuft.

Diese Seite dreht die Beweislast um. Sie sagt nicht „ich kann Docker“, sie sagt „hier ist ein System, das in Containern läuft, hier ist seine Compose-Datei, hier ist seine Uptime der letzten 91 Tage, und hier ist der Vorfall vom 14. September mit Ursache, Behebung und Post-Mortem.“

Das ist überprüfbar. Und Überprüfbarkeit ist die einzige Währung, die in diesem Kontext etwas wert ist.

### Der Preis dieser These

Sie ist unbequem. Konkret:

- Am Launch-Tag ist vieles leer. Kein Track auf `CORE`, das Betriebsraster vollständig `nodata`, der Blog ohne Eintrag. Die Seite muss das aushalten, ohne zu lügen.

- Jede Zahl braucht eine Pipeline. Uptime braucht einen externen Probe. p95 braucht Prometheus. Die Deploy-Dauer braucht eine Pipeline, die sich selbst misst.
- Ein einziger Widerspruch beschädigt alles. Steht in der Fallstudie „React Router 7“, während die Seite auf Next.js läuft, ist die These tot — nicht beschädigt, tot. Wer bei der sichtbarsten Angabe schlampft, dem glaubt man die unsichtbaren nicht.

Der letzte Punkt ist kein theoretisches Risiko. Genau dieser Fehler steckte in den Entwürfen: die Fallstudie beschrieb einen Stack, den sie nicht benutzte. Deshalb gibt es jetzt eine CI-Prüfung, die genau das verhindert (Kapitel 12).

---

## 2. Wer liest diese Seite, und was prüft er

---

Die Zielgruppe sind Hiring Manager und technische Leads. Die beiden lesen unterschiedlich, und die Seite bedient beide.

Der Hiring Manager scannt. Er ist in dreißig Sekunden entschieden, ob er weiterliest. Was er in diesen dreißig Sekunden sieht: eine Überschrift, die eine Position benennt, einen Verfügbarkeitsstatus, ein Terminal, das etwas tut, und eine Liste von Systemen mit Zuständen. Er versteht nicht jedes Detail — aber er sieht, dass hier Zahlen stehen, wo bei anderen Adjektive stehen.

Der technische Lead prüft. Er klickt auf die Fallstudie, liest die Compose-Datei, schaut sich das Betriebsraster an, sucht nach dem Punkt, an dem die Geschichte auseinanderfällt. Wenn er ihn nicht findet, ist das das stärkste Signal, das eine Bewerbung senden kann.

Für ihn ist die öffentliche API gebaut. Für ihn ist der Incident-Log da, der einen echten Ausfall mit Ursache und Behebung dokumentiert statt ihn zu verschweigen. Für ihn steht in der Fallstudie, dass der Uptime-Probe über GitHub Actions läuft und dass dessen Cron unter Last ungenau ist — eine Einschränkung, die niemand bemerkt hätte, wenn sie verschwiegen worden wäre.

Diese Ehrlichkeit ist kein moralischer Zusatz, sie ist die Funktion. Eine Seite, die ihre eigenen Schwächen benennt, wird für den Rest glaubwürdig.

---

## 3. Die neun Invarianten

---

Invarianten sind Regeln, die nicht durch Disziplin eingehalten werden, sondern durch Konstruktion. Der Unterschied ist entscheidend: Disziplin ermüdet, Konstruktion nicht.

### 1. Keine erfundenen Zahlen

Metriken sind `*float64` in Go und `number | null` in TypeScript. `null` rendert als `– NO DATA`, niemals als `0`.

Warum das im Typsystem lebt: Weil `strictNullChecks` den Compiler zwingt, den leeren Fall zu behandeln. Ein Entwickler, der eine Metrik anzeigt, kann den Fall „keine Daten“ nicht vergessen — der Build bricht. Die Regel wird nicht befolgt, sie wird erzwungen.

Ein Golden-Test sichert zusätzlich ab: für jedes System mit `state  $\neq$  'live'` muss jedes Metrikfeld `null` sein.

## 2. Skill-Zustände werden abgeleitet, nie gesetzt

Es gibt keine Spalte `tracks.state`. Der Zustand entsteht aus der Beziehung `track_evidence`:

| BELEGE IN SYSTEMEN MIT <code>STATE='LIVE'</code> | ZUSTAND               |
|--------------------------------------------------|-----------------------|
| $\geq 2$                                         | <code>core</code>     |
| genau 1                                          | <code>applied</code>  |
| 0, aber in einem System <code>in_build</code>    | <code>learning</code> |
| sonst                                            | <code>queued</code>   |

Warum keine Spalte: Weil eine Spalte gepflegt werden müsste. Und was gepflegt werden muss, driftet — spätestens in dem Moment, in dem ein System gelöscht wird und niemand daran denkt, die Zustände nachzuziehen. Als View kann der Zustand nicht falsch sein; er ist die Auswertung der Belege.

Diese Invariante ist die zerbrechlichste. Sie kippt leise: Jemand baut eine Spalte ein, weil eine Query dann schneller ist, und ein halbes Jahr später zeigt die Seite Zustände, die niemand mehr belegen kann.

## 3. Metriken nur für laufende Systeme

`queued` und `in_build` tragen niemals Uptime, p95 oder Fehlerrate. Ein System, das nicht läuft, hat keinen Betrieb — und ohne Betrieb gibt es nichts zu messen.

## 4. Ohne Post-Mortem keine Kerbe

Jeder Incident braucht `cause`, `fix` und `post_slug` als `NOT NULL`. Eine rote Zelle im Betriebsraster ohne Erklärung ist schlimmer als keine: sie sagt „hier war etwas kaputt“ und lässt den Leser raten.

Das Schema erzwingt es. Man kann keinen Vorfall eintragen, ohne ihn zu erklären.

## 5. Belege zeigen nie ins Leere

`track_evidence.system_id` hat einen Fremdschlüssel mit `ON DELETE RESTRICT`. Wird versucht, ein System zu löschen, an dem ein Beleg hängt, schlägt der Löschmodus fehl.

Warum das die wichtigste Absicherung überhaupt ist: Ein Beleg, der auf ein nicht mehr existierendes System zeigt, ist genau der Fehler, der die Seite wertlos macht. Sie würde behaupten, ein Skill sei durch ein System belegt, das es nicht gibt.

## 6. Ein Tag ohne Messung ist `nodata`, nie 100 %

Eine Lücke als Lücke zu zeigen ist die Aussage der Seite. Ein fehlender Messwert als „alles in Ordnung“ zu interpretieren wäre eine erfundene Zahl durch die Hintertür.

## 7. Das Fenster ist 91 Tage, nicht 90

13 Wochen × 7 Tage = 91. Ein Raster mit sieben Reihen fasst nur Vielfache von sieben; bei 90 hätte die letzte Spalte ein Loch.

Die Zahl steht in der Kopfzeile, in der Uptime-Kachel und in der Beschriftung — überall dieselbe, damit sie nachzählbar bleibt. Wer nachzählt und 91 findet, hat einen weiteren kleinen Beleg dafür, dass hier sorgfältig gearbeitet wurde.

## 8. Keine Farbe, kein Radius, keine Dauer außerhalb von `tokens.css`

Eine Lint-Regel verbietet Hex-Werte außerhalb der Token-Datei. Die Tailwind-Standardpalette ist abgeschaltet — funktioniert `bg-blue-500` noch, wird es irgendwann benutzt.

## 9. Genau zwei `localStorage`-Keys

`ts.theme` für die Themewahl, `ts404.best` für den Bestwert im Error-Budget-Spiel. Sonst nichts.

Das ist keine Schikane, sondern die technische Entsprechung dessen, was die Datenschutzseite verspricht. Eine Seite, die Datensparsamkeit behauptet und nebenbei fünf Schlüssel im Browser ablegt, hat wieder einen Widerspruch.

# 4. Terminal Noir – warum diese Ästhetik

Die visuelle Sprache ist kein Kostüm. Sie folgt aus der These.

Ein Terminal ist die ehrlichste Oberfläche, die es gibt. Es zeigt, was ist, ohne Zwischenschicht. Es hat keine Illustrationen, keine Beruhigungsfarben, keine abgerundeten Ecken, die Kompetenz suggerieren sollen. Es meldet Zustände. Genau das tut diese Seite auch — und deshalb sieht sie aus wie ein Terminal, statt sich nur so zu nennen.

Konkret heißt das:

Die Zustände sind Wörter, nicht Farben. `CORE`, `APPLIED`, `LEARNING`, `QUEUED` stehen als Text da. Farbe kommt hinzu, aber sie trägt die Information nie allein — auch aus Barrierefreiheitsgründen, aber vor allem, weil ein Wort präziser ist als ein Grünton.

Das Neon-Budget. Cyan ( `#00E5FF` ) liegt unter etwa 3 % der Fläche. Alert-Rot ( `#FF2D55` ) erscheint genau einmal pro Seite. Auf der Homepage ist das `YOU ARE INSIDE THIS PROJECT` an der eigenen Systemzeile. Wenn alles leuchtet, leuchtet nichts — und eine Seite, die Betriebsdisziplin behauptet, darf sich visuell keine Disziplinlosigkeit leisten.

Nummerierung, die etwas bedeutet. `SYS.01` bis `SYS.04` sind keine Dekoration. Sie sind die verbindliche Reihenfolge der Abschnitte, und die Reihenfolge trägt ein Argument: der Trainings-Log (`SYS.01`) steht vor der Systemliste (`SYS.02`), weil er die Belege liefert, auf die sich die Liste beruft. Dreht man das um, wird aus Nachweis wieder Selbstbeschreibung.

Sieben Themes, ein Standard. Terminal Noir ist verbindlich; die anderen sechs — Catppuccin Mocha, Amber CRT, Phosphor, Tokyo Night, Catppuccin Latte, Gruvbox Light — sind eine Vorliebe des Besuchers, keine Markenvariante. Ein Theme tauscht nur Farbe. Struktur, Typografie, Abstände und Bewegung bleiben. Jedes ist ein Satz von etwa zwanzig Variablen auf einem Wrapper, und jede Text- und Signalfarbe erreicht in jedem Theme mindestens 4,5:1 gegen beide Flächen.

Dass ein heller Modus dabei ist, ist wichtig: Terminal Noir ist eine Entscheidung, keine Bequemlichkeit. Wer bei `prefers-color-scheme: light` einen hellen Modus bekommt, sieht, dass hier jemand an andere Leute gedacht hat.

---

## TEIL II – DIE ARCHITEKTUR

### 5. Systemüberblick

Ein Host bei OVH, verwaltet mit Dokploy. Acht Container in zwei logischen Gruppen.

Anwendung

| CONTAINER | AUFGABE                                                                          |
|-----------|----------------------------------------------------------------------------------|
| proxy     | Traefik: TLS, Routing, Prometheus-Metriken                                       |
| web       | Next.js: Seiten, Rendering, MDX-Blog. Keine Datenlogik.                          |
| api       | Go: Datenmodell, Ableitungen, Contract, Validierung, Rate-Limit, Mail, Snapshots |
| db        | PostgreSQL 18                                                                    |

Beobachtung

| CONTAINER  | AUFGABE                                                     |
|------------|-------------------------------------------------------------|
| alloy      | Collector: scrapt Traefik und API, tait Logs, empfängt Faro |
| prometheus | Metriken, 7 Tage                                            |
| loki       | Logs, 14 Tage                                               |
| grafana    | Dashboards, Alerting                                        |

#### Warum ein Host und nicht zwei

Ein getrennter Observability-Host ist für ein Projekt verfrüht. Kommt ein zweites dazu, trägt der zweite Host drei Projekte statt eins — dann ist er richtig, und dann wird er einmal ordentlich gebaut statt zweimal halb.

Die Trennung ist vorbereitet, nicht verbaut: Alloy ist von Anfang an der Collector. Beim Umzug ändert sich nur sein Ziel von `localhost` auf `remote_write` über einen WireGuard-Tunnel. Die Anwendung merkt davon nichts.

Das ist YAGNI korrekt angewendet — nicht „brauche ich nie“, sondern „brauche ich noch nicht, und später besser“.

## Die zwei Risiken, die daraus folgen

Volle Platte legt die Datenbank lahm. Loki und Prometheus liegen auf derselben NVMe wie Postgres. Deshalb: Größen-Limits zusätzlich zur Zeit-Retention ( `retention.size=2GB` bei Prometheus, Compactor-Grenze bei Loki), eigene Volumes, Disk-Alert bei 70 %.

Der schnellste Verbraucher sind übrigens nicht die Logs, sondern alte Docker-Images — jeder Deploy legt eines an, und Docker räumt nicht von selbst auf. Ein wöchentlicher Prune-Job ist Pflicht, keine Kür.

Stirbt der Host, stirbt die Aufzeichnung mit. Das ist das ernstere Problem, und es hat eine elegante Lösung — Kapitel 8.

---

## 6. Wer welche Daten besitzt

---

Die Aufteilung ist eine bewusste Entscheidung, und sie war nicht die naheliegende.

Go besitzt die Daten. Next.js rendert.

Der naheliegende Schnitt wäre gewesen, Next.js-Route-Handler direkt auf Postgres zugreifen zu lassen und Go nur für eine Nebenaufgabe zu benutzen. Das hätte einen Netzwerk-Hop gespart. Es hätte aber die interessante Arbeit — Datenmodell, Ableitungen, Contract, Validierung — nach TypeScript verlagert und den Go-Dienst auf eine Randrolle reduziert.

Für eine Seite, deren erklärter Zweck es ist, Backend-Fähigkeit zu belegen, wäre das die falsche Richtung gewesen.

Der gewählte Schnitt bringt außerdem zwei Dinge, die zur These passen:

Ein Contract, eine Wahrheit. `openapi.yaml` erzeugt Typen auf beiden Seiten. Die Invarianten werden an genau einer Stelle durchgesetzt — im Go-Dienst. Läge die Logik in Route-Handlern, gäbe es zwei Orte, an denen sie durchgesetzt oder vergessen werden kann.

Die API ist vorzeigbar. Sie ist öffentlich lesbar und unter `/api/docs` dokumentiert. Steckte dieselbe Logik in Next.js-Handlern, wäre sie mit der Seite verwoben statt eigenständig prüfbar.

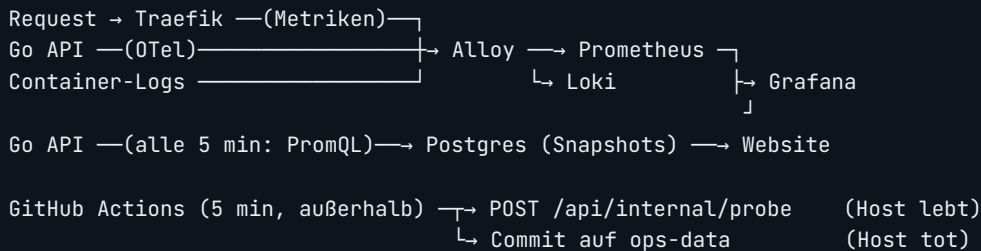
Der Preis: ein Netzwerk-Hop im internen Docker-Netz, unter einer Millisekunde. Mit den Cache Components von Next.js 16 fällt er ohnehin meist weg, weil die statische Hülle vorgerendert ist und nur Metriken frisch geholt werden.

Eine Ausnahme: der Blog. Beiträge liegen als MDX im Repo, nicht in der Datenbank. Text gehört in die Versionsverwaltung, wird über PRs reviewt und läuft durch dieselbe Pipeline wie Code. Dass eine Textänderung damit einen Deploy auslöst, ist kein Nachteil — es ist genau das, was die Fallstudie behauptet.

---

## 7. Der Messweg

Der Weg einer Zahl von ihrer Entstehung bis auf die Seite.



### Traefik misst, nicht die Anwendung

p95 und Fehlerrate kommen aus Traefiks nativen Prometheus-Metriken, nicht aus einem Zähler in der Anwendung.

Drei Gründe: Ein Zähler im Prozess ist beim Neustart weg — und deployt wird oft. Traefik sieht außerdem Requests, die die Anwendung nie erreichen (Timeouts, abgewiesene Verbindungen), und genau die interessieren, wenn etwas kaputt ist. Und es entfällt eine Klasse von Fehlern: kein Logdatei-Parsing, keine Offsets, kein Konflikt mit Dokploys nächtlicher Log-Truncation.

### Prometheus misst, Postgres serviert

Die Seite fragt nie Prometheus direkt. Stattdessen zieht die Go-API alle fünf Minuten per PromQL Snapshots und schreibt sie nach Postgres. Die Seite liest Postgres.

Warum dieser Umweg:

- Geschwindigkeit. Eine PromQL-Abfrage bei jedem Seitenaufruf wäre langsam und würde Prometheus unter Last setzen.
- Verfügbarkeit. Fällt Prometheus aus, zeigt die Seite weiter den letzten gültigen Wert mit Zeitstempel und Alter, statt zu brechen. Genau dafür ist Invariante 1 gebaut: ein alter Wert mit Datum ist ehrlich, eine Null wäre gelogen.
- Trennung der Zuständigkeiten. Prometheus ist das Messsystem, Postgres die Auslieferung. Beide dürfen unabhängig ausfallen.

## 8. Das externe Ausfallprotokoll

Ein Host kann seinen eigenen Ausfall nicht melden. Das ist keine Spitzfindigkeit, sondern das zentrale Problem jeder Uptime-Messung — und für diese Seite besonders heikel, weil Uptime eine der wenigen Zahlen ist, die sie prominent zeigt.

Läuft alles auf einem Host und fällt der Host aus, stirbt Prometheus mit der Seite. Nach dem Neustart zeigt die Uptime-Reihe eine Lücke — nicht einen Ausfall. Auf einer Seite, die Betriebsehrlichkeit zu ihrem Argument macht, ist das die falsche Art von Lücke: sie sieht aus wie fehlende Sorgfalt, nicht wie ein dokumentierter Vorfall.

## Die Lösung

Ein GitHub-Actions-Workflow prüft alle fünf Minuten von außen. Solange der Host lebt, meldet er das über `POST /api/internal/probe`. Ändert sich der Zustand, committet er zusätzlich eine Zeile auf den Datenbranch `ops-data`:

```
2026-09-14T03:11:00Z  down  connect timeout
2026-09-14T03:26:00Z  up    200 in 142ms
```

Nur bei Wechseln — ein paar Zeilen im Monat statt 288 Commits am Tag. Kommt der Host zurück, liest die API die Datei und füllt `ops_checks` rückwirkend auf. Aus der Lücke wird eine Kerbe.

## Warum das mehr ist als eine technische Lösung

Der Ausfall wird aufgezeichnet, obwohl das aufzeichnende System tot war. Und die Aufzeichnung liegt versioniert in Git — also öffentlich prüfbar, mit Zeitstempel und ohne Möglichkeit, sie stillschweigend zu korrigieren.

Das ist die These der Seite in Reinform: die Aufzeichnung des eigenen Ausfalls liegt außerhalb der eigenen Infrastruktur, damit sie den Ausfall überlebt.

Die ehrliche Einschränkung dazu, die auch auf der Seite steht: der Cron-Scheduler von GitHub Actions ist unter Last ungenau. Ein Fünf-Minuten-Takt kann in der Praxis sieben oder acht Minuten sein. Das macht die Messung nicht wertlos, aber es gehört benannt.

---

## 9. Was bewusst fehlt

Dieses Kapitel ist so wichtig wie die Architektur. Ein Portfolio, das jede Technologie einbaut, die es kennt, argumentiert schwächer als eins, das begründen kann, warum es sie weggelassen hat.

## WEGGELASSEN

## BEGRÜNDUNG

|                          |                                                                                                                                                                                                                      |
|--------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| CDN / WAF                | Würde der Datenschutzseite widersprechen. Mit OVH als Registrar, DNS, Host und Mail steht keine dritte Partei im Anfrageweg. Schutz am Origin: Rate-Limit, fail2ban, Firewall — OVH filtert Netzebenen-DDoS ohnehin. |
| Kubernetes               | Vier Anwendungscontainer auf einem Host. k3s vor dem Launch bringt null Leser; ein Blogeintrag danach bringt welche.                                                                                                 |
| Redis                    | Postgres und In-Process-Cache reichen bei diesem Verkehr.                                                                                                                                                            |
| Mimir                    | Das „M“ in LGTM ist horizontal skalierbarer Mehrmandanten-Speicher. Ein Host, ein Dienst, ein Nutzer — ein Prometheus reicht.                                                                                        |
| Sentry (self-hosted)     | 40+ Container, 8–16 GB RAM. Und BSL-lizenziert, also kein OSI-Open-Source.                                                                                                                                           |
| WebGL / Three.js         | ~150 KB gzip für eine Ambient-Ebene bei 6 % Deckkraft — bricht das eigene Constraint „schnell auf dem Handy im Mobilfunknetz“. Canvas-2D leistet dasselbe.                                                           |
| Session Replay           | Auf einer Portfolioseite gibt es keine Nutzerflüsse zu rekonstruieren.                                                                                                                                               |
| Storybook                | Die Route <code>/dev/components</code> leistet dasselbe ohne zweiten Build.                                                                                                                                          |
| Feature Flags, A/B-Tests | Ein Entwickler, eine Zielgruppe, kein Experimentbedarf.                                                                                                                                                              |

Zwei Dinge sind trotzdem drin, obwohl ihr Nutzen überwiegend demonstrativ ist — und das gehört ehrlich gesagt:

SBOM, Signierung und SLSA-Provenance schützen einen Solo-Betrieb praktisch kaum. Sie kosten zusammen etwa fünfzehn Zeilen Workflow, und „mein Image ist signiert und die Herkunft ist öffentlich prüfbar“ ist ein Absatz, den fast kein Bewerber schreiben kann. Der Aufwand-Nutzen-Schnitt geht auf; der Nutzen liegt nur woanders, als man denkt.

Chaos-Drills wären für einen Portfolio-Betrieb Overkill — außer dass der Incident-Log auf der Fallstudie echten Inhalt braucht. Ohne Drills müsste man auf einen echten Ausfall warten. Doppelter Zweck, deshalb drin. # TEIL III — DIE MECHANIK

## 10. Das Datenmodell

Zehn Tabellen und eine View. Das Modell ist klein, weil die Invarianten in ihm stecken statt in Anwendungscode.

```

systems —< track_evidence >— tracks — modules
      |               |               |
      |               |               | (v_track_states: View)
      |               |               |
      |< ops_checks   |               |
      |< ops_days >— incidents
      |< deploys
      |< metric_snapshots

contact_messages  (steht allein)

```

### systems

Der Kern. Jedes System hat zwei unabhängige Achsen, und die Trennung ist wichtig:

**state** beschreibt den Betrieb: **live**, **in\_build**, **queued**. **source** beschreibt den Codezugang: **public** (URL Pflicht) oder **private** (Grund Pflicht, **nda** oder **internal**).

Ein System kann öffentlich sein und nicht laufen. Es kann laufen und geschlossen sein. Diese Fälle in ein Feld zu quetschen wäre der klassische Modellierungsfehler — man merkt ihn erst, wenn der erste NDA-Kunde kommt.

Ein Check-Constraint erzwingt die Kopplung:

```

CHECK ((source_access='public' AND source_url IS NOT NULL)
      OR (source_access='private' AND source_reason IS NOT NULL))

```

◊ **PRIVATE · NDA** ist eine ehrliche Angabe. ◊ **PRIVATE** ohne Grund wäre eine Ausrede — deshalb lässt das Schema sie nicht zu.

### tracks, modules, track\_evidence

22 Tracks in fünf Modulen: Languages, Backend, Data, DevOps, Foundations.

**tracks** hat keine Zustandsspalte. Der Zustand entsteht in **track\_evidence** — der Tabelle, die einen Track mit einem System verbindet und dazu einen Detailtext trägt:

```

track_id → tracks.id
system_id → systems.id      ON DELETE RESTRICT
detail    "BUILD + DEPLOY"

```

Daraus wird die Belegzeile, die unter jedem Track steht:

```

CI/CD (GitHub Actions)                                APPLIED
SHIPPED IN → 02 TIMSEIL.DEV (BUILD + DEPLOY)

```

Ohne Beleg lautet sie **NO SYSTEM YET → SELF-STUDY**. Das ist keine Schwäche, sondern eine Aussage: hier wird gelernt, aber noch nichts betrieben.

`ops_checks`, `ops_days`, `incidents`, `deploys`

Rohdaten und Aggregat getrennt: `ops_checks` sammelt Einzelmessungen, `ops_days` ist das Tagesaggregat, aus dem das Raster rendert. `incidents` hängen an Tagen, `deploys` am System.

`incidents.cause`, `.fix` und `.post_slug` sind `NOT NULL` — Invariante 4 als Schemaregel.

`metric_snapshots`

Die alle fünf Minuten aus Prometheus gezogenen Werte. Getrennt von `ops_days`, weil sie eine andere Herkunft und eine andere Lebensdauer haben.

`contact_messages`

Name, E-Mail, Nachricht, Zeitpunkt, IP nur als Hash, Zustellstatus. Die einzige Tabelle mit personenbezogenen Daten — und die einzige, die von der öffentlichen API niemals gelesen wird.

## Zwei Datenbankrollen statt einer

`timseil_migrate` besitzt das Schema und darf DDL. `timseil_app` darf nur DML auf die Tabellen, die es braucht.

Der Grund ist konkret: Eine SQL-Injection in der API kann dann kein Schema löschen. Das ist Least Privilege an der Stelle, an der es am billigsten und am wirksamsten ist.

## 11. Die Ableitung

Das Herzstück. Wenn ein Leser dieses Buchs nur ein Kapitel behält, sollte es dieses sein.

```
CREATE VIEW v_track_states AS
SELECT t.id, t.module_id, t.name,
       COUNT(DISTINCT s.id) FILTER (WHERE s.state='live') AS live_count,
       CASE
         WHEN COUNT(DISTINCT s.id) FILTER (WHERE s.state='live') ≥ 2 THEN 'core'
         WHEN COUNT(DISTINCT s.id) FILTER (WHERE s.state='live') = 1 THEN 'applied'
         WHEN COUNT(DISTINCT s.id) FILTER (WHERE s.state='in_build') > 0 THEN 'learning'
         ELSE 'queued'
       END AS state
FROM tracks t
LEFT JOIN track_evidence te ON te.track_id = t.id
LEFT JOIN systems s       ON s.id = te.system_id
GROUP BY t.id, t.module_id, t.name;
```

### Was die Schwellen bedeuten

`core` verlangt zwei laufende Systeme. Das ist streng, und die Strenge ist der Punkt. Etwas einmal gebaut zu haben heißt, es einmal zum Laufen gebracht zu haben. Es zweimal in Betrieb zu haben heißt, die Fälle kennengelernt zu haben, die beim ersten Mal Glück waren.

Am Launch-Tag steht deshalb kein einziger Track auf `core`. 22 Tracks, davon 13 `applied` aus timseil.dev, 9 `learning` ohne System. Die Kopfzeile sagt: `SELF-TRACKED · 22 TRACKS · EVIDENCE: 01 SYSTEM`.

Das sieht schwach aus. Es ist das Gegenteil: eine Seite, die bei sich selbst streng ist, ist glaubwürdig, wenn sie über andere Dinge spricht.

## Die zeitliche Achse

Der Log ist kein Zustand, sondern eine Bewegung. Kommt ein System dazu, wandern die Tracks, die es benutzt, von `learning` auf `applied`; ihre Belegzeile nennt das neue System. Erreicht ein Track zwei laufende Systeme, springt er auf `core`.

Das macht die Seite zu etwas, das wächst, statt zu etwas, das gepflegt wird. Ein klassisches Portfolio veraltet ab dem Tag der Veröffentlichung. Dieses wird mit jedem System genauer.

## Die Absicherung

Dieselbe Logik existiert zweimal: als SQL-View und als `skillState()` in `tokens.ts` aus dem Design-Handoff. Ein Property-based Test erzeugt tausend zufällige Belegkonstellationen und prüft, dass beide dasselbe liefern.

Warum das nötig ist: Zwei Implementierungen derselben Regel driften. Der Test macht die Drift zu einem Build-Fehler statt zu einem stillen Widerspruch zwischen Frontend und Backend.

## 12. Metriken – nullable by design

Die Regel ist einfach, die Umsetzung interessant.

```
type SystemMetrics struct {
    Uptime90d *float64 `json:"uptime90d"`
    P95Ms     *float64 `json:"p95Ms"`
    ErrorRate *float64 `json:"errorRate"`
}
```

```
type SystemMetrics = {
    uptime90d: number | null
    p95Ms:     number | null
    errorRate: number | null
}
```

Der Zeiger in Go und `| null` in TypeScript sind dieselbe Aussage: dieser Wert kann fehlen, und du musst das behandeln.

Mit `strictNullChecks` ist das keine Bitte. Wer `metrics.p95Ms.toFixed(0)` schreibt, bekommt einen Compile-Fehler. Er muss den leeren Fall behandeln, und der leere Fall ist `– NO DATA`.

## Die drei Zustände einer Kachel

| ZUSTAND | WANN             | ANZEIGE                           |
|---------|------------------|-----------------------------------|
| ok      | Wert vorhanden   | Zahl in <code>tabular-nums</code> |
| no-data | null             | – NO DATA                         |
| stale   | letzter Wert alt | Wert + Zeitstempel + Retry-Zähler |

Der dritte ist der interessante. Fällt Prometheus aus, zeigt die Seite nicht nichts und auch nicht die letzte Zahl so, als wäre sie frisch. Sie zeigt die Zahl mit ihrem Alter. Das ist die ehrlichste verfügbare Aussage.

### Warum 0 verboten ist

0 und „keine Messung“ sehen in einer Zahl gleich aus, bedeuten aber das Gegenteil. Eine Fehlerrate von 0 % ist ein hervorragender Wert. Eine fehlende Fehlerrate ist gar kein Wert. Sie gleich darzustellen wäre die eleganteste Art, unbemerkt zu lügen — und genau deshalb steht das Verbot in den Invarianten und nicht in einem Styleguide.

## 13. Das Betriebsraster

Das Bauteil, das den Unterschied zwischen Aktivität und Betrieb sichtbar macht.

Der Contribution-Graph misst Aktivität — wie viel jemand committet. Das ist bekanntlich manipulierbar und sagt wenig.

Das Betriebsraster misst Betrieb — ob ein System lief. Eine Zelle pro Tag, eine Kerbe pro Vorfall, ein Balken pro Deploy.

Beide stehen nebeneinander auf der Homepage, und die Sektion nennt für jeden Block seine Quelle. Der Vergleich ist das Argument: der eine Graph zeigt, dass jemand gearbeitet hat, der andere, dass etwas funktioniert.

### Zwei Ausführungen

| ORT                                    | FENSTER                   | VERHALTEN                       |
|----------------------------------------|---------------------------|---------------------------------|
| Homepage <code>SYS.03</code>           | 30 Tage, einreihig        | reine Anzeige, keine Klickziele |
| Fallstudie <code>.04 OPERATIONS</code> | 91 Tage Desktop, 30 mobil | Kerben anklickbar → Vorfall     |

Die Homepage zeigt einen Streifen, die Fallstudie das Raster. Die anklickbaren Kerben liegen dort, wo auch Platz für die Erklärung ist.

## Die vier Zustände

`ok` · `degraded` · `outage` · `nodata`

Der vierte ist der wichtigste. Ein Tag ohne Messung ist grundsätzlich `nodata` — niemals `ok`. Am Tag 1 ist das gesamte Raster `nodata`, mit einer Zeile darunter, die erklärt, dass es sich pro Betriebstag füllt.

Ein leeres Raster ist ehrlicher als ein volles, das erfunden wurde. Und es hat einen Nebeneffekt: wer nach drei Monaten wiederkommt, sieht den Unterschied.

## Warum 91 und nicht 90

Sieben Reihen fassen nur Vielfache von sieben.  $13 \times 7 = 91$ . Bei 90 hätte die letzte Spalte ein Loch, das aussähe wie ein Fehler.

Die Zahl steht in Kopfzeile, Kachel und Beschriftung identisch — damit sie nachzählbar bleibt. Bei 91 Tagen ist das Raster 243 Pixel breit und passt auch auf einem 390er-Display ohne horizontales Scrollen.

## Die Post-Mortem-Pflicht

Jede Kerbe öffnet Dauer, Fehlerbudget, Ursache, Behebung und den Log-Eintrag. Ohne Post-Mortem keine Kerbe — Schema-Constraint, nicht Vorsatz.

Der Grund: eine rote Zelle ohne Erklärung ist schlimmer als keine. Sie sagt „hier war etwas kaputt“ und überlässt dem Leser die Interpretation. Mit Erklärung wird aus demselben Vorfall ein Beleg für Betriebsreife.

# 14. Der Contract

`contract/openapi.yaml` ist die einzige Wahrheit über die Schnittstelle. Aus ihr werden Typen für beide Seiten generiert:

```
openapi.yaml └─> oapi-codegen      → api/internal/httpx/gen.go
               └─> openapi-typescript → web/lib/api/schema.d.ts
```

Niemand schreibt einen Typ von Hand, der im Contract steht. `make gen` erzeugt beide, und CI prüft mit `git diff --exit-code`, dass nichts uncommitted ist — Contract-Drift bricht den Build.

## Die Endpoints

| ENDPOINT                               | ZWECK                                             |
|----------------------------------------|---------------------------------------------------|
| GET <code>/api/health</code>           | Liveness, Version, Commit-SHA, Ops-Metriken       |
| GET <code>/api/systems</code>          | Liste mit Zustand, Quelle, nullable Metriken      |
| GET <code>/api/systems/{slug}</code>   | Detail + Incidents + Deploys + 91-Tage-Raster     |
| GET <code>/api/training</code>         | 5 Module, 22 Tracks, abgeleitete Zustände, Belege |
| GET <code>/api/contributions</code>    | GitHub-Graph, serverseitig gecacht                |
| POST <code>/api/contact</code>         | Formular, rate-limited                            |
| POST <code>/api/internal/probe</code>  | Token, externer Uptime-Check                      |
| POST <code>/api/internal/deploy</code> | Token, Pipeline meldet ihre Dauer                 |

Die internen Endpoints sind doppelt geschützt: Token mit konstantzeitigem Vergleich, und zusätzlich blockt Traefik den Pfad von außen. Sie stehen nicht in `/api/docs`.

### Warum die Dokumentation öffentlich ist

`/api/docs` rendert den Contract über Scalar — für jeden lesbar. Das ist Absicht: Es senkt die Hürde für den technischen Leser, der prüfen will, von „ich müsste raten, wie diese API aussieht“ auf „ich klicke drauf“.

Eine API, die man nicht findet, ist als Beleg wertlos.

### Casing

Die API liefert `lowercase` ( `core`, `live`, `applied` ), die Oberfläche zeigt `UPPERCASE`. Die Umwandlung passiert in der Darstellung, nie in den Daten. Sonst hätte man zwei Schreibweisen in der Datenbank und irgendwann einen Vergleich, der stillschweigend fehlschlägt.

## 15. Der Contribution-Graph

Ein kleines Bauteil mit drei nicht offensichtlichen Details.

Die REST-API von GitHub liefert den Contribution-Kalender nicht. Der Weg ist GraphQL:

```

query($login: String!) {
  user(login: $login) {
    contributionsCollection {
      contributionCalendar {
        totalContributions
        weeks { contributionDays { date contributionCount contributionLevel } }
      }
    }
  }
}

```

Das kostet erfahrungsgemäß eine Stunde Suche, wenn man es nicht weiß.

Das Token bleibt serverseitig. Scope `read:user`, kein `NEXT_PUBLIC_`-Präfix, Aufruf nur in der Go-API. Ein Token im Browser wäre ein Fund für jeden, der die Netzwerkanfragen ansieht.

GitHubs Farben werden nicht verwendet. Die API liefert `contributionLevel` von `NONE` bis `FOURTH_QUARTILE` — das passt 1:1 auf die fünf Stufen des Entwurfs. Die mitgelieferten `color`-Werte sind GitHub-Grün und würden das Neon-Budget sprengen. Verwendet werden die eigenen Cyan-Stufen `--10` bis `--14`.

Bei Ausfall: Cache mit Alter. Serverseitig eine Stunde gecacht, Circuit Breaker davor. Antwortet GitHub nicht, zeigt die Seite den letzten guten Stand mit dem Hinweis „aus Cache, N h alt“ — dasselbe Muster wie bei den Metriken.

---

## 16. Das Kontaktformular

---

Der einzige Konversionspunkt der Seite, und der einzige unauthentifizierte Schreibzugriff. Entsprechend viel steckt darin.

### Warum kein `mailto:`

Ein `mailto:`-Link scheitert stumm auf jedem Gerät ohne eingerichtetes Mailprogramm — Firmenlaptop, Windows mit Chrome, Handy mit Gmail im Browser. Der Besucher klickt, nichts passiert, er geht weg. Bei dem einen Element, das eine Bewerbung auslösen könnte, ist das nicht akzeptabel.

### Die TX-Spur

Rechts neben dem Formular baut sich live der Request auf, den es senden wird:

```

POST /api/contact HTTP/2
host: timseil.dev
content-type: application/json

{
  "name": "...",
  "email": "...",
  "message": "...",
  "company": "", ← honeypot, muss leer bleiben
  "dwellMs": 4018,
  "ts": "2026-08-16T16:54:18Z"
}

```

Das ist der thematisch dichteste Moment der Seite: Sie erklärt, was sie tut, während sie es tut. Und es beantwortet die Frage „wo geht das hin?“ ohne ein Wort Erklärtext.

## Spam-Abwehr ohne CAPTCHA

| MITTEL                        | WIRKUNG                                                                             |
|-------------------------------|-------------------------------------------------------------------------------------|
| Honeypot <code>company</code> | Per CSS versteckt, nie <code>display:none</code> am Label — sonst erkennen Bots es  |
| Verweildauer < 3 s            | Verwerfen ohne Rückmeldung                                                          |
| Rate-Limit                    | 3 Anfragen pro IP in 10 Minuten, IP nur als Hash                                    |
| Idempotenz                    | <code>ts</code> + <code>email</code> + Nachrichten-Hash — Doppelklick sendet einmal |

Kein CAPTCHA. Es verlagert Arbeit auf den Besucher, um ein Problem zu lösen, das drei Zeilen serverseitig auch lösen.

## Die Mail-Header-Injection

Der Sicherheitsfund, der beim Durchgang auffiel und leicht übersehen wird.

`Reply-To` wird aus der Besucheradresse gebaut. Enthält die ein `\r\n`, hängt ein Angreifer eigene Header an und macht aus dem Formular einen Spam-Relay auf der eigenen Domain — mit der Reputation, die gerade mit SPF und DKIM aufgebaut wurde.

Gegenmittel: CRLF hart ablehnen, Adresse strikt prüfen, Mail als Plaintext bauen, nie als HTML.

## Die Regel, die aus dem Provider folgt

OVH MX Plan verlangt, dass das `From:`-Feld dem SMTP-Konto entspricht, mit dem authentifiziert wird. Also:

```

From:      contact@timseil.dev      ← immer
Reply-To:  <Adresse aus dem Formular>

```

Das deckt sich zufällig mit dem Entwurf — jetzt ist auch klar, dass es die einzig mögliche Lösung ist.

## Was nach dem Absenden passiert

Erfolg gibt `202` mit einer Message-ID. Der Text bleibt im Feld stehen. Das ist eine bewusste Entscheidung gegen die Konvention: die ID ist der Beleg, den der Absender zitieren kann, und der Text geht nicht verloren, wenn er ihn noch braucht.

Im Fehlerfall stehen Code, Versuchszähler und die Adresse als Ausweg da. Der Text geht nie verloren.

## 17. Das Terminal

Die Signature-Komponente — und die am häufigsten missverstandene.

### Es ist ein Befehlsregister, keine Shell

Eine statische Map von Befehl auf Handler. Jeder Handler gibt Zeilen zurück. Nichts, was ein Besucher tippt, wird ausgeführt.

Ein echtes Terminal im Browser hieße PTY auf dem Server plus WebSocket für stdin und stdout — so arbeiten ttyd, Wetty oder GoTTY. Und es hieße: wer diesen Endpoint erreicht, hat Shell-Zugriff auf den VPS. Auf einer öffentlichen Seite ist das kein abwägbares Risiko, sondern ein unbedingtes Nein.

### Simulierte Oberfläche, echte Daten

Die entscheidende Unterscheidung. Die Shell-Metapher ist Bedienoberfläche, die Ausgaben sind real:

| BEFEHL                                    | QUELLE                                |
|-------------------------------------------|---------------------------------------|
| <code>whoami</code>                       | Operator-Profil                       |
| <code>stack</code>                        | dieselbe Quelle wie der Trainings-Log |
| <code>projects</code> · <code>work</code> | <code>/api/systems</code>             |
| <code>blog</code>                         | MDX-Index                             |
| <code>cv</code>                           | lädt die PDF direkt                   |
| <code>contact</code>                      | Kanäle                                |
| <code>help</code> · <code>clear</code>    | statisch                              |
| <code>matrix</code>                       | undokumentiert                        |

Dass `stack` und der Trainings-Log dieselbe Quelle lesen, ist testbar: ändert man einen Track-Zustand, ändert sich die `stack`-Ausgabe mit. Das steht als Abnahmekriterium im Bauplan.

## Der `cv`-Weg

Es gibt keine CV-Seite, keinen Download-Knopf, keinen Nav-Eintrag und keine Route. Der Befehl `cv` lädt die PDF direkt.

Das ist die konsequenteste Design-Entscheidung der Seite: Der Lebenslauf ist hinter einer Interaktion versteckt, die zeigt, dass man das Terminal benutzt hat. Wer ihn findet, hat sich mit der Seite beschäftigt.

Mobil ist das Terminal read-only, deshalb verlinkt die Fußzeile dort `CV → cv.pdf` direkt — als echtes `<a href download>`. Eine Anweisung, die auf dem Gerät nicht ausführbar ist, wäre eine Sackgasse.

## Die sieben Sicherheitsregeln

Auch ein simuliertes Terminal hat eine Angriffsfläche:

1. Kein `eval`, kein `new Function()`, kein dynamischer Import aus Eingabe. Unbekannter Befehl → fester Text.
2. Ausgabe ist Daten, nie HTML. Zeilen sind `{ text, tone }`, `tone` ein Enum auf einen Token. Nie `dangerouslySetInnerHTML` — sonst ist der Echo eines unbekannten Befehls ein XSS-Vektor. Der Testfall: `<img src=x onerror=alert(1)>` muss eine Textzeile erzeugen, kein Popup.
3. Längen deckeln: Eingabe ~200 Zeichen, Buffer ~500 Zeilen.
4. Jeder Befehl zeigt auf einen festen Endpoint. Keine Eingabe fließt in eine URL. Kein `curl`-artiger Befehl — das wäre ein offener Proxy.
5. `cv` lädt einen festen Pfad. Nie ein Pfadsegment aus Eingabe.
6. Keine Debug-Befehle. Kein `env`, kein `version` über das hinaus, was `/api/health` ohnehin zeigt. Genau hier leaken solche Spielereien.
7. `matrix` respektiert `prefers-reduced-motion` und ist jederzeit abbrechbar.

## Barrierefreiheit

Eingabe mit `aria-label`, Ausgabe in einer `aria-live="polite"`-Region, Terminal zuletzt in der Tab-Reihenfolge. Bei mehr als etwa zehn Ausgabezeilen geht eine Zusammenfassung in die Live-Region und der Rest als normaler Text darunter — sonst liest ein Screenreader fünfzig Zeilen am Stück vor.

## 18. Das Error-Budget-Spiel

Auf der 404-Seite, und das einzige rein spielerische Element der Seite. Es verdient trotzdem ein Kapitel, weil es das Konzept der Seite in eine Mechanik übersetzt.

## Was es ist

Ein `<canvas>` mit vier Spuren: `API`, `DB`, `QUEUE`, `CACHE`. Anfragen laufen auf den Spuren nach unten, man bedient sie mit `A S D F` oder den Pfeiltasten, Trefferfenster  $\pm 10\%$  um den Handler-Balken.

Das HUD zeigt `SERVED`, einen live gerechneten p95, die Uptime und fünf Kästchen Error-Budget. Jede verpasste Anfrage kostet ein Kästchen. Sind alle fünf weg, endet das Spiel mit `PAGED` und einer Incident-Zusammenfassung.

## Warum das mehr ist als ein Osterei

Es lehrt in dreißig Sekunden, was ein Fehlerbudget ist: ein Kontingent an Fehlern, das man ausgeben darf, bevor es Konsequenzen gibt. Wer das gespielt hat, versteht die Zahlen auf der Fallstudie anders.

Und es schließt einen Kreis: die Seite definiert echte SLOs mit echtem Fehlerbudget. Die fünf Kästchen im Spiel sind dieselbe Idee — nur schneller.

## Technische Details, die zählen

Der Spielzustand liegt in Instanzfeldern, nicht in React-State. Eine rAF-Schleife, die sechzigmal pro Sekunde ein Re-Render auslöst, wäre eine Katastrophe für die Performance. Nur der Bestwert wird persistiert: `localStorage["ts404.best"]`, ein Schlüssel, sonst nichts.

Es startet nie von selbst und blockiert nichts. Die 404-Seite funktioniert vollständig ohne das Spiel — Router-Trace, montierte Routen, Rückwege. Das Spiel ist Zugabe.

Barrierefreiheit: Das Canvas ist fokussierbar ( `tabindex="0"`, `role="application"`, `aria-label` ) und per Tastatur spielbar. Das Ergebnis nach `PAGED` erscheint zusätzlich als Text in einer `aria-live`-Region — sonst stünde der Zustand nur im Canvas und wäre für einen Screenreader unsichtbar. Bei `prefers-reduced-motion` startet es nur auf ausdrückliche Eingabe. # TEIL IV — DIE OBERFLÄCHE

# 19. Tokens und das Theme-System

---

## Eine Quelle für jeden Wert

`tokens.css` ist verbindlich. Die Regel steht als Kommentar in der Datei selbst: keine Farbe, kein Radius, keine Dauer außerhalb dieser Datei.

Die Kernpalette von Terminal Noir:

| TOKEN                      | WERT                 | KONTRAST GEGEN <code>--BG</code> |
|----------------------------|----------------------|----------------------------------|
| <code>--bg</code>          | <code>#0A0E14</code> | —                                |
| <code>--panel</code>       | <code>#0E141C</code> | —                                |
| <code>--ink</code>         | <code>#E8EEF4</code> | 16,55                            |
| <code>--steel</code>       | <code>#8B98A6</code> | 6,58                             |
| <code>--dim</code>         | <code>#7C8996</code> | 5,41 — Untergrenze für Text      |
| <code>--acc</code> (Cyan)  | <code>#00E5FF</code> | 12,57                            |
| <code>--alert</code> (Rot) | <code>#FF2D55</code> | 5,30 — ein Moment pro Seite      |
| <code>--amber</code>       | <code>#FFB000</code> | 10,56                            |

Die Kontrastwerte stehen als Kommentar neben jedem Token. Das ist eine kleine Sache mit großer Wirkung: Wer eine Farbe ändern will, sieht sofort, ob er unter 4,5 fällt. Barrierefreiheit wird damit zur Eigenschaft der Datei statt zu einem späteren Prüflauf.

### Tailwind wird abgeleitet, nicht ergänzt

```
@import "tailwindcss";
@import "./tokens.css";

@theme inline {
  --color-bg:      var(--bg);
  --color-panel:   var(--panel);
  --color-ink:     var(--ink);
  --color-acc:     var(--acc);
  --font-mono:     var(--mono);
  --font-display:  var(--display);
}
```

Danach wird die Tailwind-Standardpalette abgeschaltet. Funktioniert `bg-blue-500` noch, wird es irgendwann benutzt — meist um zwei Uhr nachts, wenn etwas schnell gehen muss.

### Drei Schriften mit drei Rollen

| FAMILIE        | ROLLE                                          |
|----------------|------------------------------------------------|
| JetBrains Mono | Labels, Daten, Zustände, Terminal, alle Zahlen |
| Chakra Petch   | Display, Überschriften — kantig, technisch     |
| Geist          | Fließtext                                      |

Geladen über `next/font/google`, damit zur Laufzeit kein Request an fonts.gstatic.com geht. Das ist nicht nur Performance: Es ist die Voraussetzung dafür, dass die Datenschutzseite stimmt, wenn sie sagt, dass keine dritte Partei im Anfrageweg steht.

Dreizehn Größenstufen, keine halben Pixel. Abstände nach einem 4er-Raster: 6, 8, 10, 12, 14, 16, 20, 26, 34, 44, 56, 72, 96. Sektionsabstand 96 px auf dem Desktop, 64 px mobil.

## Sieben Themes

Terminal Noir ist der Standard. Daneben Catppuccin Mocha, Amber CRT, Phosphor, Tokyo Night, Catppuccin Latte, Gruvbox Light.

Ein Theme tauscht nur Farbe. Struktur, Typografie, Abstände, Bewegung und Regeln bleiben: ein Akzent, ein Alert-Moment pro Seite, Akzentfläche unter etwa 3 %, Zustand nie nur über Farbe.

Technisch ist jedes Theme ein Satz von etwa zwanzig Variablen auf einem `[data-theme]`-Wrapper.

Ohne gespeicherte Wahl folgt die Seite dem System: `prefers-color-scheme: light` liefert Gruvbox Light, sonst Terminal Noir.

## Das Anti-Flash-Problem

Ein Theme, das erst nach dem ersten Paint gesetzt wird, erzeugt ein sichtbares Umspringen. Deshalb ein Inline-Script im `<head>`, vor jedem CSS:

```
<script>(function(){try{var t=localStorage.getItem('ts.theme');
  if(t)document.documentElement.dataset.theme=t;}catch(e){}})();</script>
```

Das kollidiert mit der Content Security Policy. Eine CSP ohne `unsafe-inline` blockt genau dieses Script. Die Lösung ist ein Nonce, den `proxy.ts` pro Request erzeugt und in beide Stellen einsetzt. Wer das übersieht, hat entweder eine schwache CSP oder ein flackerndes Theme — beides vermeidbar.

---

## 20. Layout und die sieben Prüfbreiten

---

### Die Zahlen kommen aus dem Inhalt

Breakpoints stehen dort, wo ein konkretes Element bricht — nicht bei Gerätenamen.

| BEREICH     | WAS PASSIERT                          | HERLEITUNG                                                                                                                         |
|-------------|---------------------------------------|------------------------------------------------------------------------------------------------------------------------------------|
| $\geq 1440$ | Spalte bei 1160, Ränder wachsen       | Entwürfe auf 1160 gezeichnet                                                                                                       |
| 1440–1080   | <code>min(1160px, 100% - 80px)</code> | dazwischen wird interpoliert, nicht umgebaut                                                                                       |
| $< 1080$    | fünf Zweispalter lösen sich auf       | Spec-Rail 400 + Abstand 80 + Lesemaß 517 = 1077, aufgerundet — und bewusst über 1024, damit ein iPad quer nicht im Grenzfall liegt |
| $< 900$     | Kopf → Menüknopf, Hero einspaltig     | Nav, Umschalter und Uhr brauchen ~520 px                                                                                           |
| $< 720$     | alles einspaltig, H1 auf 34           | darunter bleibt für die zweite Spalte $< 300$ px                                                                                   |
| $< 560$     | Tabellenzeilen werden Karten          | sechs Rasterspalten ergäben 60-px-Spalten                                                                                          |
| 390         | Referenzbreite mobil, Rand 22 px      | kleinstes gezeichnetes Gerät                                                                                                       |

Die 1077 ist das schönste Beispiel: eine Zahl, die aus drei Messungen entsteht statt aus einer Konvention. Wer bei 1024 bräche, hätte ein iPad quer im Grenzfall.

### Getestet wird an sieben Breiten

1440 · 1081 · 1079 · 1024 · 899 · 719 · 390 — jeder Schalter beidseitig.

Der Grund für die Paare 1081/1079 und die Nähe: Ein Breakpoint, der nur einseitig getestet wird, ist nicht getestet. Fehler passieren an der Kante.

#### `pointer: coarse` statt Breite

Die 44-px-Regel für Trefferflächen und das read-only-Terminal hängen an `pointer: coarse`, nicht an der Fensterbreite. Ein Tablet mit 1000 px bekommt große Ziele, ein schmales Desktop-Fenster nicht.

Das ist der korrekte Umgang mit dem Unterschied zwischen „klein“ und „Touch“ — zwei Dinge, die oft verwechselt werden.

### Kein Hover auf Touch

Auf Mobil ist der Hover-Zustand der Ruhezustand: Vorschauen sichtbar, Zeilen bei 100 % Deckkraft, Terminal read-only mit sichtbarer Befehlsliste. Informationen, die auf dem Desktop hinter Hover liegen, sind auf Touch nicht versteckt, sondern von vornherein da.

## 21. Die Boot-Sequenz

2400 Millisekunden, sechs Frames, einmal pro Session. Der erste Eindruck.

| ZEIT    | FRAME           | WAS                                                                    |
|---------|-----------------|------------------------------------------------------------------------|
| 0       | FIRST PAINT     | Schwarz, ein Caret. Kein Logo, kein Spinner.                           |
| 80 ms   | PANEL SNAP      | Bracket-Ecken fahren von 1,5× auf 1× zusammen — ein Snap, kein Fade    |
| 340 ms  | INIT LOG        | Sieben Zeilen à 100 ms, ganz — nicht buchstabenweise                   |
| 640 ms  | HEADLINE DECODE | Text löst sich aus Zufallszeichen, 40 ms je Zeichen, links nach rechts |
| 1040 ms | MODULE CASCADE  | Skill-Zeilen mit 60 ms Versatz, Graph-Spalten mit 12 ms                |
| 2400 ms | STEADY STATE    | Ruhezustand. Danach bewegt sich nur, was etwas meldet.                 |

### Die Regel, die alles trägt

Kein Layout-Shift. Die Seite ist ab Frame 0 vollständig gelayoutet — nur unsichtbar. Animiert werden ausschließlich `opacity`, `transform` und `clip-path`. Die Headline belegt ihren Platz von Anfang an; nichts springt.

Das ist der Unterschied zwischen einer Intro-Animation und einem Systemstart. Ein Intro baut auf, ein Systemstart macht sichtbar, was schon da ist.

Messbar: CLS = 0.

### Die dokumentierte Falle

Aus dem Design-Handoff, und sie kostet sonst einen Abend:

Der dekodierte Text muss der Komponente gehören, die ihn anzeigt. Teilt die Headline den Render-Zyklus mit dem Init-Log, wird ihr Textknoten mitten im Scramble ersetzt — und der Endzustand erscheint nie.

Also: eigene Komponente mit eigenem State. Verbindlich bleiben die 640 ms, nicht der Prototyp-Woraround, der den Start auf 1700 ms schob, um das Problem zu umgehen.

### Abbrechbar und einmalig

Klick, Taste oder Scroll springt sofort auf den Ruhezustand. Ein `sessionStorage`-Flag sorgt dafür, dass der zweite Besuch im selben Tab direkt im Endzustand startet.

Niemand wartet zweimal auf eine Animation. Wer das nicht einbaut, verwandelt einen guten ersten Eindruck in einen schlechten zweiten.

## Das Neon-Budget im Start

In der Sequenz ist nur die Uplink-Zeile, sind die Bracket-Ecken und die Graph-Füllung cyan. Kein Alert-Rot beim Start. Rot ist für einen Moment reserviert, und der Systemstart ist keiner.

## Reduced Motion

Bei `prefers-reduced-motion: reduce` erscheint sofort der Endzustand, das Init-Log vollständig, ohne Puls und ohne Ambient. Keine reduzierte Fassung der Animation — gar keine.

## 22. Scroll-Choreografie

| ELEMENT       | VERHALTEN                         |
|---------------|-----------------------------------|
| Header-Wipes  | bei <code>entry 12%</code>        |
| Skill-Zeilen  | 60 ms Versatz                     |
| Graph-Spalten | 12 ms je Spalte                   |
| Parallax      | Ruler 0,4× · Grid 0,15×           |
| Spec-Rail     | sticky bei <code>top: 96px</code> |

Scroll-gekoppelte Bewegungen laufen rückwärts, zeitbasierte nicht. Das ist die Regel, die den Unterschied zwischen „reagiert auf mich“ und „läuft ab“ ausmacht.

## Die Fallback-Frage

Die Scroll-Kopplung nutzt `animation-timeline: view()`. Firefox unterstützt das nicht — der Baseline-Status ist „Limited availability“.

Die Lösung ist Kapselung in `@supports`:

```
@supports (animation-timeline: view()) {
  .reveal { animation: wipe linear both; animation-timeline: view();
    animation-range: entry 12% entry 60%; }
}
```

Ohne Unterstützung greift die Regel nicht, und `animation-fill-mode: both` lässt das Element im Endzustand. Firefox sieht alles, nur ohne die scroll-gekoppelte Feinheit. Für den Pin reicht das nicht — dort übernimmt GSAP ScrollTrigger.

## Lenis und GSAP dürfen sich nicht streiten

Beide wollen die Scroll-Position kontrollieren. Läuft Lenis parallel zu ScrollTrigger, ruckelt es sichtbar.

Die Lösung: Lenis füttert ScrollTrigger — `ScrollTrigger.update` im Lenis-Callback. Einer führt, einer folgt.

## Der eine Pin

`SYS.02` ist als gepinnter Moment gedacht, der über 1,4 Viewports mehrere Systeme durchschaltet. Mit zwei Systemen trägt er nicht und bleibt bis zum dritten deaktiviert — ein Pin, der zwei Zeilen durchschaltet, sieht nach ungenutzter Mechanik aus.

Unter 900 px entfällt er ohnehin.

## 23. Zustandssprache

Ein eigenes Referenzblatt im Design, und der Grund, warum die Seite über elf Seiten konsistent wirkt.

### Zwei Wörter, die leicht verwechselt werden

| WORT                   | BEDEUTUNG                | ORT                     |
|------------------------|--------------------------|-------------------------|
| <code>AVAILABLE</code> | Verfügbarkeit der Person | Hero der Startseite     |
| <code>ONLINE</code>    | Auslieferung der Seite   | Meta-Leiste jeder Seite |

Beide standen ursprünglich als `ONLINE` da. Ein Konsistenzlauf über elf Seiten fand den Widerspruch — die Seite hätte behauptet, ein Mensch sei „online“.

### Jeder Zustand hat ein zweites Merkmal

Nie nur Farbe. Immer zusätzlich Ecken, Rahmen, Unterstrich oder Wortlaut.

Der Grund ist doppelt: Barrierefreiheit für Farbfeldsichtige, aber auch Präzision. `LEARNING` ist eindeutiger als ein Gelbton, den jeder anders interpretiert.

### Leerzustände sind Erstklassenbürger

Für jede Liste, jedes Raster und jede Metrik existiert ein definierter Leerzustand — und er wird zuerst gebaut, nicht zuletzt.

Das ist die praktische Konsequenz aus der These: Eine Seite, die am Launch-Tag vieles leer zeigt, muss die Leere gestaltet haben. Ein nachträglich eingebauter Leerzustand sieht immer aus wie ein Fehler.

## Fehlerzustände erklären

Bei einem Ausfall zeigt eine Kachel den letzten gültigen Wert, den Zeitstempel und den Retry-Zähler. Nicht nichts, nicht eine Null, nicht ein Spinner ohne Ende.

---

## 24. Barrierefreiheit

---

Kein Anhängsel, sondern eine Eigenschaft der Tokens und Bauteile.

- Kontrast: Alle Textfarben erreichen AA in allen sieben Themes; die Werte stehen als Kommentar im Token. Die 28-%-Ruhezustände der Skill-Zeilen sind dekorativ gedimmt — die Information steht zusätzlich als Zustandswort.
  - Fokus: `outline: 1px solid #00E5FF; outline-offset: 3px`, nur `:focus-visible`, überall dieselbe Form.
  - Tab-Reihenfolge: Leserichtung, Skip-Link zuerst, Terminal zuletzt.
  - Formular: `<label for>`, Fehler über `aria-describedby`, `aria-invalid`, Fokus aufs erste Fehlerfeld, Sendestatus in `aria-live`. Der Honeypot bekommt `aria-hidden="true"` und `tabindex="-1"` — sonst stolpert ein Screenreader-Nutzer hinein und wird als Bot verworfen.
  - Theme-Wahl: `role="radiogroup"`, `aria-checked`; der aktive Knopf trägt volle Deckkraft und einen Akzentrahmen.
  - Touch: Jedes Ziel  $\geq 44 \times 44$  — nachgemessen, nicht gegrept. Der erste Prüflauf zählte nur `min-height:44px` und übersah 17 Chips, deren Höhe aus Innenabstand entsteht.
  - Mono 9 ist die Untergrenze; darunter trägt kein Element Inhalt.
  - `<html lang="en">`.
-

## TEIL V – DER BETRIEB

### 25. Container und Compose

Die Startreihenfolge ist keine Zierde — sie steht als Ausschnitt in der Fallstudie und muss deshalb stimmen:

```
db (pg_isready)
  ↳ migrate (Init-Container, läuft durch)
    ↳ api (Healthcheck /api/health)
      ↳ web
```

Jede Stufe wartet auf die Gesundheit der vorherigen. `depends_on` mit `service_healthy` beziehungsweise `service_completed_successfully`.

#### Bilder

Go: Multi-Stage, `CGO_ENABLED=0`, `-trimpath`, Ldflags mit Version und Commit-SHA, `distroless/static:nonroot`, read-only rootfs, Base-Image per Digest gepinnt. Unter 20 MB.

Next.js: `output: 'standalone'` auf `node:24-alpine`, non-root.

Die Standalone-Falle: `.next/standalone` enthält weder `public/` noch `.next/static`. Beide müssen explizit kopiert werden, sonst fehlen im Container alle Assets und Schriften. Das ist der häufigste Fehler beim Self-Hosting von Next.js — und er äußert sich als „lokal geht’s, im Container nicht“.

#### Gebaut wird in CI, nie auf dem Host

Es hängt an einer Zeile:

```
image: ghcr.io/g1ng4r/timseil-api:${IMAGE_TAG} #
# build: ./api #
```

Drei Gründe, der dritte ist der wichtigste:

1. RAM — ein Next.js-Build zieht kurzzeitig 2–4 GB, genau während des Deploys.
2. Disk — der Build-Cache belegt mehrere GB auf einer 40-GB-Platte.
3. Verifizierbarkeit — baut man auf dem Host, ist das getestete Artefakt nicht das laufende. Der Contract-Test lief gegen ein anderes Image, und die Signatur gilt für eines, das nie deployed wurde. Auf dieser Seite bricht damit die ganze Kette.

Abnahmekriterium: `docker system df` auf dem Host zeigt 0 B Build-Cache.

## Named Volumes, keine Bind Mounts

Dokploys Volume-Backups nach S3 funktionieren nur mit Docker Named Volumes. Ein Bind Mount lässt sich nicht sichern — und das merkt man typischerweise an dem Tag, an dem man ein Backup braucht.

## 26. CI/CD

Sieben Schritte, wie sie auch auf der Fallstudie stehen:

```
01 PUSH → 02 LINT → 03 TEST → 04 BUILD
→ 05 PUSH IMG → 06 DEPLOY → 07 VERIFY
```

Am Ende meldet die Pipeline ihre eigene Dauer an `POST /api/internal/deploy`. Damit ist die Zahl auf der Seite gemessen, nicht behauptet.

### Rollback

Das vorherige Image-Tag wird gemerkt. Liefert `/api/health` nach sechzig Sekunden kein `200`, rollt der Deploy zurück und der Job schlägt fehl.

Das muss einmal wirklich provoziert werden — ein Rollback, der nie ausgelöst wurde, ist eine Vermutung. Es steht als Abnahmekriterium im Bauplan.

### Zero-Downtime mit Compose

Compose kann kein Rolling Update wie Swarm. Der Weg:

```
docker compose up -d --no-deps --scale api=2 --wait api
docker compose up -d --no-deps --scale api=1 --wait api
```

Zweite Instanz hoch, Healthcheck abwarten, Traefik nimmt sie auf, alte Instanz runter. Das setzt Graceful Shutdown voraus — sonst schneidet das Herunterskalieren laufende Anfragen ab.

Falls es nicht sauber gelingt: dann steht in der Fallstudie „~3 s geplante Downtime pro Deploy“ statt „Zero-Downtime“. Die Zahl muss stimmen.

### Migrations abwärtskompatibel

Expand/Contract. Rollt der Code zurück, rollt das Schema nicht mit — ein Deploy, der eine Spalte löscht, macht den Rollback unmöglich.

## Lieferkette

`govulncheck`, `npm audit`, Trivy auf Images, `gitleaks` über History und Diff, `gosec`, CodeQL. Dazu Syft für das SBOM, cosign für die keyless Signatur über Sigstore und GitHub-Actions-Attestation für SLSA-Provenance.

Die Pipeline selbst ist gehärtet: Jeder Workflow bekommt `permissions:` explizit — `contents: read` als Standard, `packages: write` nur beim Push, `id-token: write` nur für cosign. Fremde Actions sind auf Commit-SHA gepinnt, nicht auf Tags — ein Tag lässt sich verschieben, und genau so lief der `tj-actions`-Angriff. Eine kompromittierte Action würde sonst ein fremdes Image mit dem eigenen Schlüssel signieren.

## 27. Observability

### Messen und Auswerten sind zwei Dinge

Messen läuft ab Tag 1, weil Zeitreihen sich nicht rückwirkend erzeugen lassen: strukturierte Logs, Traefik-Metriken, Prometheus, Loki, der externe Probe, die Snapshots nach Postgres.

Auswerten kommt danach: OTel-Traces, Tempo, Dashboards, Burn-Rate-Alerts, Faro. Dashboards für leere Zeitreihen zu bauen wäre Arbeit ins Nichts.

### Der Collector

Grafana Alloy — die OpenTelemetry-Collector-Distribution von Grafana. Ein Binary für Metriken, Logs, Traces und den Faro-Empfänger.

Promtail ist nicht dabei: seit dem 2. März 2026 End-of-Life. Wer es heute einbaut, baut auf einer Komponente ohne Sicherheitsupdates.

### Frontend-Telemetrie ohne zusätzlichen Dienst

Fehler mit Stacktrace, Web Vitals und Breadcrumbs kommen über das Faro Web SDK in den `faro.receiver` von Alloy — also in eine Komponente, die ohnehin läuft. Kein extra Container, kein extra RAM.

Source Maps werden erzeugt, aber nicht öffentlich ausgeliefert. Alloy löst sie von der Platte auf. Ein minimierter Stacktrace wäre wertlos; öffentliche Source Maps wären ein Geschenk an jeden, der die Anwendung analysieren will.

Der Release wird über den Commit-SHA zugeordnet — denselben, der im Image-Tag und in `/api/health` steht. Damit beantwortet ein Fehler die Frage „seit welchem Deploy?“ von selbst.

## Warum kein Sentry

Self-hosted Sentry braucht 40+ Container und 8–16 GB RAM. Es ist außerdem BSL-lizenziert, also kein OSI-Open-Source. Sentry als SaaS würde bedeuten, Fehlerberichte mit URLs, IPs und Nutzerkontext an einen Dritten zu schicken — auf einer Seite, die sich bewusst gegen ein CDN entschieden hat, ein Widerspruch.

Was tatsächlich fehlt, ist Gruppierung und ein Issue-Workflow. Bei drei bis fünf verschiedenen Fehlern über Monate ist eine LogQL-Query in Grafana die Gruppierung. Wächst das Volumen, kommt GlitchTip (MIT, 512 MB) — dann mit Anlass statt als Reflex.

## 28. SLOs und Fehlerbudget

| SLI            | MESSUNG                          | SLO               | BUDGET / 30 D |
|----------------|----------------------------------|-------------------|---------------|
| Verfügbarkeit  | externer Probe, 5-min-Takt       | 99,5 %            | 3 h 39 min    |
| Latenz Seiten  | Traefik p95                      | < 300 ms bei 99 % | —             |
| Latenz API     | Traefik p95, <code>/api/*</code> | < 150 ms bei 99 % | —             |
| Fehlerrate     | 5xx / alle Requests              | < 0,1 %           | —             |
| Zustellbarkeit | erfolgreiche Formularendungen    | > 99 %            | —             |

### Warum 99,5 % und nicht 99,9

99,9 % wären 43 Minuten Budget im Monat. Auf einem einzelnen VPS mit Sicherheitsupdates und Deploys ist das unrealistisch — und ein SLO, das man reißt, ist schlimmer als keins.

99,5 % sind 3 Stunden 39 Minuten. Das ist erreichbar, und es lässt Raum für einen echten Vorfall, der dann dokumentiert wird, statt versteckt zu werden.

Ein SLO ist ein Versprechen, das man halten kann. Alles andere ist Marketing.

### Burn-Rate-Alerts

Nach dem Muster aus dem Google-SRE-Workbook: ein schnelles Fenster (1 h, 14,4-fache Verbrennung) meldet sofort, ein langsames (6 h, 6-fach) erzeugt ein Ticket.

Der Sinn: Ein Alert bei jedem einzelnen Fehler ist Lärm. Ein Alert, wenn das Monatsbudget in einer Stunde zu einem Viertel verbrannt wird, ist ein Signal.

Zu jedem Alert gehört ein Runbook. Ein Alert ohne erprobtes Runbook zählt nicht — um drei Uhr nachts nützt eine Meldung ohne Anleitung nichts.

## Der Kreis zum Spiel

Die fünf Kästchen im Error-Budget-Spiel auf der 404-Seite sind dieselbe Idee. Wer gespielt hat, versteht die Uptime-Zahl auf der Fallstudie anders — nicht als Note, sondern als Kontostand.

## 29. Sicherheit

### Die Angriffsfläche

| FLÄCHE                         | EXPONIERT               | SCHUTZ                                   |
|--------------------------------|-------------------------|------------------------------------------|
| Next.js, API-Lesezugriffe      | öffentlich, gewollt     | CSP, Header, Rate-Limit                  |
| <code>POST /api/contact</code> | öffentlich              | Honeypot, Dwell-Time, Rate-Limit, Origin |
| Terminal                       | öffentlich, keine Shell | Befehlsregister, Ausgabe als Daten       |
| <code>/api/internal/*</code>   | zu                      | Traefik blockt plus Token                |
| Dokploy-UI                     | zu                      | nur über SSH-Tunnel                      |
| Prometheus, Loki, Alloy        | zu                      | keine Traefik-Labels, keine Ports        |
| Postgres                       | nie                     | nur Docker-Netz                          |
| SSH                            | Key                     | fail2ban, kein Passwort, kein Root       |
| OVH-Konto                      | —                       | 2FA — sonst Totalkompromittierung        |

### Die zwei ersten Punkte

Die Dokploy-Oberfläche hat vollen Zugriff auf Host, Deploys und alle Umgebungsvariablen — also auf sämtliche Secrets. Sie ist das lohnendste Ziel der ganzen Maschine. Deshalb: kein Traefik-Router, Port in der Firewall zu, Zugriff nur über `ssh -L 3000:localhost:3000`. Der Komfortverlust ist ein Kommando.

Prometheus, Loki und Alloy haben standardmäßig keine Authentifizierung. Prometheus kann über seine Admin-API Zeitreihen löschen; Loki liefert sämtliche Logs. Solange sie hinter einem WireGuard-Tunnel auf einem zweiten Host lagen, war das folgenlos. Auf einem gemeinsamen Host ist dieser Schutz weg und muss ersetzt werden: keine Labels, keine Port-Mappings, nur `expose:` im Docker-Netz.

Abnahme: `nmap` von außen zeigt ausschließlich 22, 80, 443.

### Backups, die man nicht löschen kann

Die S3-Zugangsdaten liegen auf dem Host. Wer den Host übernimmt, löscht sonst auch die Sicherungen — das klassische Ransomware-Muster.

Gegenmittel: Bucket-Versionierung und Object-Lock, oder ein Schlüssel mit `PutObject` und `ListBucket`, aber ohne `DeleteObject`. Kostet nichts und ist der Unterschied zwischen Vorfall und Totalverlust.

Abnahmekriterium: Ein Löschmodversuch mit dem Produktionsschlüssel muss scheitern.

## Kein CDN

Der Handoff verlangt „DNS ohne Proxy davor, damit die Aussage der Datenschutzseite stimmt“. Mit OVH als Registrar, DNS-Betreiber, Host und Mail-Anbieter steht keine dritte Partei im Anfrageweg.

Der Preis, benannt: kein WAF, die VPS-IP ist sichtbar. Aufgefangen am Origin — Rate-Limit in Traefik und in der API, fail2ban, Firewall, und OVHs Netzebenen-Filter.

Ein Portfolio, das seinen Ausfallschutz an einen CDN delegiert, argumentiert schwächer als eins, das ihn selbst löst und erklärt.

## Das Threat Model kommt vor der Härtung

Ein Threat Model, das nach der Härtung entsteht, dokumentiert nur, was ohnehin gebaut wurde. Davon gebaut, steuert es sie. Deshalb steht die schlanke Fassung — eine Stunde STRIDE über die Container — vor den Härtungsphasen, die ausführliche danach als Blogeintrag.

# 30. Datenschutz

Die Datenschutzseite ist kein Formalakt, sondern eine Behauptung wie jede andere auf dieser Seite — und sie muss deshalb stimmen.

| ZUSAGE                            | UMSETZUNG                                                   |
|-----------------------------------|-------------------------------------------------------------|
| Access-Logs 14 Tage               | Rotation konfiguriert                                       |
| Anwendungslogs 7 Tage             | Loki-Retention                                              |
| Rate-Limit-IP 10 Minuten          | nur im Speicher, nur als Hash                               |
| Keine dritte Partei im Anfrageweg | kein CDN, Schriften selbst gehostet, Umami self-hosted      |
| Kein Tracking, kein Cookie        | Umami cookieless, zwei <code>localStorage</code> -Schlüssel |

Der Code muss einhalten, was die Seite verspricht. Eine Retention-Regel, die nur auf der Datenschutzseite steht und nicht in der Konfiguration, ist eine Unwahrheit mit Rechtsfolgen.

## Was das Formular ändert

Ohne Formular verarbeitet die Seite praktisch keine personenbezogenen Daten. Mit Formular: Name, E-Mail, Nachricht, Zeitpunkt, IP-Hash fürs Rate-Limit, Mail-Provider als Auftragsverarbeiter. Dazu die Fehlererfassung über Faro.

Das gehört in die Datenschutzseite, samt Löschprozess. In Luxemburg unter der DSGVO ist das keine Kür.

---

## TEIL VI – REFERENZ

### A. Endpoint-Referenz

| METHODE | PFAD                              | ÖFFENTLICH | ANTWORT                                      |
|---------|-----------------------------------|------------|----------------------------------------------|
| GET     | <code>/api/health</code>          | ja         | Version, SHA, Ops-Metriken                   |
| GET     | <code>/api/systems</code>         | ja         | Systeme, Zustand, Quelle, nullable Metriken  |
| GET     | <code>/api/systems/{slug}</code>  | ja         | Detail, Incidents, Deploys, 91-Tage-Raster   |
| GET     | <code>/api/training</code>        | ja         | Module, Tracks, abgeleitete Zustände, Belege |
| GET     | <code>/api/contributions</code>   | ja         | Contribution-Graph, gecacht                  |
| GET     | <code>/api/docs</code>            | ja         | Scalar-Rendering des Contracts               |
| POST    | <code>/api/contact</code>         | ja         | 202 / 400 / 429 / 502                        |
| POST    | <code>/api/internal/probe</code>  | nein       | Token + Traefik-Block                        |
| POST    | <code>/api/internal/deploy</code> | nein       | Token + Traefik-Block                        |

### B. Zustands-Referenz

System: `live` · `in_build` · `queued` Track: `core` · `applied` · `learning` · `queued` Betriebstag: `ok` · `degraded` · `outage` · `nodata` Deploy: `ok` · `rollback` Quelle: `public` (URL Pflicht) · `private` (Grund Pflicht)

API liefert `lowercase`, Oberfläche zeigt `UPPERCASE`.

### C. Glossar

Ableitung — Zustände entstehen aus Beziehungen, nicht aus Feldern. Kern der These.

Beleg (Evidence) — Verbindung zwischen einem Track und einem System, in dem er läuft.

Fehlerbudget — Anteil an Fehlern, den ein SLO zulässt, bevor es verletzt ist. Bei 99,5 % sind das 3 h 39 min im Monat.

Kerbe — Markierung im Betriebsraster für einen Vorfall. Existiert nur mit Post-Mortem.

Neon-Budget — Regel, dass Akzentfarbe unter etwa 3 % der Fläche bleibt und Alert-Rot einmal pro Seite erscheint.

`nodata` — Ein Tag ohne Messung. Nie `ok`, nie 100 %.

Prüfstein — `curl /api/systems` liefert dieselben Zahlen wie die Seite.

Snapshot — Alle fünf Minuten aus Prometheus gezogener Wert, in Postgres für die Auslieferung.

Terminal Noir — Der Standard unter sieben Themes; verbindlich für die Struktur.

Trainings-Log — `SYS.01`. 22 Tracks in fünf Modulen mit abgeleiteten Zuständen und Belegzeilen.

## D. Entscheidungsregister

| NR   | ENTSCHEIDUNG                 | KERN                                             |
|------|------------------------------|--------------------------------------------------|
| 0001 | Next.js statt React Router   | Handoff-Vorgabe; Entwürfe werden korrigiert      |
| 0002 | Blog als MDX im Repo         | Text gehört in die Versionsverwaltung            |
| 0003 | Zustände als SQL-View        | Was gepflegt werden muss, driftet                |
| 0004 | API öffentlich lesbar        | Der Prüfstein der These                          |
| 0005 | Go besitzt die Daten         | Backend-Fähigkeit ist der Zweck der Seite        |
| 0006 | Kein CDN                     | Datenschutzseite bleibt wahr                     |
| 0007 | Prometheus statt Log-Parsing | Übersteht Neustarts, misst mehr                  |
| 0008 | Ein Host zum Launch          | Zweiter Host, wenn ein zweites Projekt ihn trägt |

## Nachwort

Diese Seite ist eine Wette darauf, dass Überprüfbarkeit überzeugender ist als Behauptung.

Die Wette kann verlieren. Ein Besucher, der nur dreißig Sekunden bleibt, sieht vielleicht nur eine dunkle Seite mit vielen `– NO DATA`. Das Argument entfaltet sich erst für den, der genauer hinsieht — und nicht jeder tut das.

Aber die Alternative ist eine weitere Seite mit Prozentbalken, die niemand glaubt und die nichts unterscheidet. Zwischen einer Seite, die von manchen ernst genommen wird, und einer, die von allen überflogen wird, ist die erste die bessere Wahl.

Was die Seite wirklich stark macht, steht nicht im Code, sondern in der Zeit. Am Launch-Tag ist sie eine sorgfältig gebaute Hülle mit einem laufenden System. Nach dem zweiten System springen die ersten Tracks auf `CORE`. Nach drei Monaten füllt sich das Betriebsraster. Nach dem ersten echten Vorfall steht dort ein Post-Mortem, das niemand erfinden konnte.

Sie ist nicht fertig, wenn sie live geht. Sie fängt dann erst an zu argumentieren.